

MSB

⚠ **Security Warning:** Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

Challenge Description

This image passes LSB statistical analysis, but we can't help but think there must be something to the visual artifacts present in this image...

Hints

- What's causing the 'corruption' of the image?
-

Tools Used

- `file` – Identify image file type and properties
 - `strings` – Extract readable text from binary files
 - `exiftool` – Inspect image metadata
 - `binwalk` – Detect embedded files and hidden data
 - `zsteg` – Detect steganography in PNG/BMP images (LSB analysis)
 - `Python` – Custom script for MSB extraction
 - `PIL/Pillow` – Python imaging library for pixel manipulation
-

Complete Solution

Step 1: Download and Prepare

Download the image named `Ninja-and-Prince-Genji-Ukiyoe-Utagawa-Kunisada.flag.png` from the challenge page.



佐野喜



The image appears to be a beautiful Japanese ukiyo-e print, but with visible "corruption" or visual artifacts.

Step 2: Basic File Inspection

First, verify the file type and general information:

```
file Ninja-and-Prince-Genji-Ukiyoe-Utagawa-Kunisada.flag.png
```

Expected output:

```
Ninja-and-Prince-Genji-Ukiyoe-Utagawa-Kunisada.flag.png: PNG image data, 1074 x 1500, 8-bit/color RGB, non-interlaced
```

Step 3: Search for Visible Text

The flag may be present as plain text. Try extracting all character strings:

```
strings Ninja-and-Prince-Genji-Ukiyoe-Utagawa-Kunisada.flag.png | grep -i "picoCTF"
```

Output: (no results)

⚠ No flag appears yet. We need to dig deeper.

Step 4: Inspect Metadata

Use `exiftool` to examine the file metadata:

```
exiftool Ninja-and-Prince-Genji-Ukiyoe-Utagawa-Kunisada.flag.png
```

Complete output:

```
ExifTool Version Number      : 13.50
File Name                    : Ninja-and-Prince-Genji-Ukiyoe-Utagawa-
Kunisada.flag.png
```

```
Directory          : .
File Size          : 3.4 MB
File Modification Date/Time : 2026:05:15 14:31:26+02:00
File Access Date/Time      : 2026:05:15 14:31:39+02:00
File Inode Change Date/Time : 2026:05:15 14:31:36+02:00
File Permissions       : -rw-rw-rw-
File Type           : PNG
File Type Extension    : png
MIME Type           : image/png
Image Width          : 1074
Image Height         : 1500
Bit Depth            : 8
Color Type           : RGB
Compression          : Deflate/Inflate
Filter               : Adaptive
Interlace            : Noninterlaced
Image Size           : 1074x1500
Megapixels           : 1.6
```

Nothing suspicious in the metadata.

Step 5: Scan with Binwalk

Use `binwalk` to check for any embedded files:

```
binwalk Ninja-and-Prince-Genji-Ukiyoe-Utagawa-Kunisada.flag.png
```

Output:

DECIMAL	HEXADECIMAL	DESCRIPTION
0	0x0	PNG image, 1074 x 1500, 8-bit/color RGB, non-interlaced
41	0x29	Zlib compressed data, default compression
1908849	0x1D2071	JB00T STAG header, image id: 0, timestamp 0xFE7EAF6C, image size: 3988551582 bytes, image JB00T checksum: 0x8F4D, header JB00T checksum: 0xFC3D

A `JB00T` header appears, but nothing immediately obvious.

Step 6: Analyze with zsteg (LSB Detection)

Since standard LSB detection tools like `zsteg` are designed for Least Significant Bit steganography, let's run it anyway:

```
zsteg Ninja-and-Prince-Genji-Ukiyoe-Utagawa-Kunisada.flag.png
```

Output:

```
imagedata          .. text: "~~~|{|}}}"
chunk:0:IHDR        .. file: Adobe Photoshop Color swatch, version 0, 1074 colors; 1st
RGB space (0), w 0x5dc, x 0x802, y 0, z 0; 2nd space (32768), w 0x8000, x 0x8080, y
0x80, z 0
b1,g,lsb,xy        .. file: Common Data Format (Version 2.5 or earlier) data
b1,g,msb,xy        .. file: Common Data Format (Version 2.5 or earlier) data
b2,r,lsb,xy        .. text: ["U" repeated 8 times]
b2,g,lsb,xy        .. file: Matlab v4 mat-file (little endian)
\252\252\252\252\252\252\252\252, numeric, rows 4294967295, columns 4294967295
b2,g,msb,xy        .. file: Matlab v4 mat-file (little endian) UUUUUUUU, numeric,
rows 4294967295, columns 4294967295
b2,b,lsb,xy        .. text: ["U" repeated 8 times]
b4,r,lsb,xy        .. text: ["w" repeated 8 times]
b4,r,msb,xy        .. text: ["U" repeated 12 times]
b4,g,msb,xy        .. text: ["w" repeated 16 times]
b4,b,lsb,xy        .. text: "\"\"\"\"\"\"\"\"\"\"4DC\""
b4,b,msb,xy        .. text: "wwwwwwwww3333"
```

Nothing looks like a flag. The hint mentions “visual artifacts” – this suggests the corruption might be caused by modifications to the **Most Significant Bits (MSB)** rather than LSB.

Step 7: Extract MSB with Python Script

The corruption visible in the image is likely due to data hidden in the **Most Significant Bits (MSB)** of the RGB channels. Let's write a Python script to extract these bits:

```
from PIL import Image
import re
```

python

```
def extract_msb_flag(image_path):
    """
    Extracts Most Significant Bits (MSB - bit 7) from a PNG image
    and searches for the picoCTF flag.
```

MSB (bit 7) is the most significant. Modifying this bit causes visible changes in the image (hence the "visual artifacts").

```
"""
```

```
# Open the image
```

```
img = Image.open(image_path)
```

```
pixels = img.load()
```

```
width, height = img.size
```

```
bits = ''
```

```
# Iterate through all pixels
```

```
for y in range(height):
```

```
    for x in range(width):
```

```
        r, g, b = pixels[x, y][:3] # Ignore alpha if present
```

```
        # Extract bit 7 (MSB) from each channel
```

```
        # (r >> 7) & 1: shift right by 7 bits and isolate the last bit
```

```
        bits += str((r >> 7) & 1)
```

```
        bits += str((g >> 7) & 1)
```

```
        bits += str((b >> 7) & 1)
```

```
# Convert bits to ASCII text (8-bit chunks)
```

```
text = ''
```

```
for i in range(0, len(bits) - len(bits) % 8, 8):
```

```
    byte = bits[i:i+8]
```

```
    text += chr(int(byte, 2))
```

```
# Search for the flag
```

```
match = re.search(r'picoCTF\[([^\]]+)\]', text)
```

```
if match:
```

```
    return match.group(0)
```

```
else:
```

```
    # Print a preview for debugging
```

```
    print("[*] Flag not found. Extracted text preview:")
```

```
    print(text[:500])
```

```
    return None
```

```
# Execute
```

```
flag = extract_msb_flag('Ninja-and-Prince-Genji-Ukiyoe-Utagawa-Kunisada.flag.png')
```

```
print(flag)
```

Output:

💡 Python Code Explanation

Element	Explanation
<code>PIL.Image.open()</code>	Opens the PNG image file
<code>pixels[x, y]</code>	Accesses RGB values at pixel position (x, y)
<code>(r >> 7) & 1</code>	Shifts the most significant bit (bit 7) to position 0 and isolates it
Why MSB?	The hint mentions “corruption” and “visual artifacts” – modifying MSB visibly alters the image, unlike LSB (which is invisible)
<code>chr(int(byte, 2))</code>	Converts an 8-bit binary string to an ASCII character
<code>re.search()</code>	Searches for the pattern <code>picoCTF{...}</code> in the extracted text

🔧 Variations to Try (If Extraction Fails)

If direct extraction doesn’t work, here are some variations to try:

Issue	Solution
Different channel order	Try <code>(g >> 7) & 1, (r >> 7) & 1, (b >> 7) & 1</code> (GBR) or <code>(b >> 7) & 1, (g >> 7) & 1, (r >> 7) & 1</code> (BGR)
Row vs Column traversal	Swap loops: <code>for x in range(width): for y in range(height):</code>
Data in a single channel	Extract only one channel: <code>bits += str((r >> 7) & 1)</code>

Issue	Solution
Use LSB instead of MSB	Replace <code>>> 7</code> with <code>& 1</code> (LSB extraction)
Alpha channel data (RGBA)	Check if image has alpha: <code>pixels[x, y][3]</code>
Different bit plane	Try bits 0-7: <code>(r >> n) & 1</code> where <code>n = 0 to 7</code>

 Final Result

picoCTF{<REDACTED>}

Note: The actual flag is redacted here. In the real challenge, running the Python script will reveal the complete flag.

 Key Concepts

Concept	Description
MSB (Most Significant Bit)	The highest-order bit in a byte (bit 7). Modifying it causes visible changes to the image.
LSB (Least Significant Bit)	The lowest-order bit (bit 0). Modifying it is invisible to the human eye.
Visual Artifacts	Visible corruption in an image often indicates MSB steganography.
RGB Channels	Red, Green, Blue – each pixel has three color channels, each storing one byte (8 bits).
MSB Extraction	Isolating the highest bit of each color channel to reconstruct hidden data.

Concept	Description
Steganography Types	LSB = invisible, MSB = visible (creates artifacts).
PIL/Pillow	Python library for image processing and pixel manipulation.
Bit Plane	Each of the 8 bits in a byte represents a different “plane” (bit 0 = lowest, bit 7 = highest).

Additional Resources

You can also use online tools to analyze steganography in images:

- [AperiSolve](#) – Online steganography analysis tool
- [StegOnline](#) – Upload and analyze images online
- [sigBits](#) – Command-line tool for bit-level steganography

Note: Since this image is large (3.4 MB), online tools may take time to process. The Python script is the most efficient method.

Pro Tips

1. **MSB vs LSB** – If an image looks “corrupted” or has visible artifacts, suspect **MSB** steganography (not LSB).
2. **Visual inspection** – The image quality degradation is a clue that the most significant bits have been altered.
3. **Try different channel orders** – Some tools store data in GBR or BGR order instead of RGB.
4. **Check all bit planes** – Data can be hidden in any of the 8 bit planes (bit 0 to bit 7).
5. **Online tools have limits** – Large images may be rejected or slow. Python scripts are more reliable for big files.
6. **Always check the hint** – “Visual artifacts” directly points to MSB manipulation.